

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)**Department of Computer Science and Engineering****CO3 ASSESSMENT TOOL 2 – Code Review & Repository Evaluation**

Course Code:	CSA10 / CSA1063	Course Title:	Software Engineering
CO Assessed:	CO3	Assessment:	Assessment Tool 1 – Code Review & Repository Evaluation
Weightage:	20%	Total Marks:	25
CO3 Statement:	Build full-stack applications with an emphasis on containerization, version control, and collaborative development		
Student Name:	v.tejaswini	Reg. No.:	192424360
Date:	17-08-2026	Duration:	60 minutes

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Annexure A – Code Review & Repository Evaluation

Instructions to Students:

Each student has been assigned an individual problem statement. Based on the assigned problem statement, perform a Code Review and Repository Evaluation of your project. Analyze the quality of the source code, repository organization, documentation, coding practices, and overall maintainability. Provide appropriate screenshots, code snippets, diagrams, and justifications wherever applicable.

Assignment Questions

Annexure A – Code Review & Repository Evaluation

1. Problem Overview

a) Explain the assigned problem statement and summarize the implemented solution.

The assigned project focuses on developing a software solution for the given problem statement by applying proper software engineering and development practices. The system is designed to identify the major requirements of the problem and provide an efficient, reliable, and user-friendly solution. The implemented application contains the required modules and functionalities based on the project objectives. The source code is organized into appropriate files and folders so that each functionality can be developed and maintained separately. The solution follows a structured development approach and uses suitable programming technologies, databases, and supporting tools. The application processes user inputs, performs the required operations, and produces meaningful outputs. Testing is performed to verify that the implemented features work correctly. Git is used to maintain different versions of the source code, while the repository stores the complete project files and documentation. Overall, the implemented solution addresses the identified problem and provides the required functionality in an organized manner.

b) Describe the project objectives and key functionalities.

Project Objectives:

- To develop a reliable solution for the assigned problem.
- To provide an easy-to-use and efficient application.
- To implement the required functional modules.
- To maintain source code using Git version control.
- To improve code quality, maintainability, and collaboration.
- To test and validate the application before deployment.

Key Functionalities:

- User interaction and input processing.

- Data storage and retrieval.
- Processing and validation of user data.
- Execution of the main application functions.
- Generation and display of appropriate results.
- Error handling and input validation.
- Testing and validation of application features.

2. Repository Organization

a) Evaluate the repository structure, folder organization, and file naming conventions.

The repository is organized using a logical folder structure in which related files are grouped together. Source code, configuration files, documentation, testing files, and other project resources are maintained separately wherever applicable. Meaningful folder and file names are used to make the purpose of each file easy to understand. The main project files are placed in the appropriate directories, avoiding unnecessary duplication and confusion. Configuration and dependency files are maintained separately from application source code. Documentation such as the README file is placed at the repository root so that developers can understand the project quickly. Test files are organized separately to simplify testing and maintenance. The naming convention is kept consistent throughout the project. This structure makes the repository easier to navigate and reduces the time required to locate specific files.

Example Repository Structure:

Project-Repository/

```

|
|— src/
|   |— main.py
|   |— modules/
|   |   └— services/
|
|— tests/
|   |— test_main.py
|   |— test_modules.py
|
|— docs/

```

```
|  └─ project_documentation.md
|
|  └─ requirements.txt
|  └─ README.md
|  └─ .gitignore
|  └─ Dockerfile
```

b) Justify how the repository layout supports maintainability and collaboration.

The repository layout supports maintainability by separating different components of the project into clearly defined directories. Developers can modify one module without unnecessarily affecting other modules. A standardized naming convention makes files easier to identify and reduces confusion among team members. The separation of source code and test files allows developers to test changes systematically. The README provides project information to new contributors, while the dependency file helps them configure the development environment. Git allows multiple developers to work on different parts of the project using branches and merge their changes later. Therefore, the organized repository structure improves readability, maintainability, collaboration, and future development.

3. Code Quality Review

a) Review the source code for readability, modularity, coding standards, and documentation.

The source code was reviewed based on readability, modularity, coding standards, and documentation. Meaningful variable and function names are used to make the code understandable. The application logic is divided into functions or modules instead of keeping all operations in a single large block of code. Proper indentation and consistent formatting improve code readability. Comments and documentation are included where the purpose of complex operations needs clarification. Reusable functions reduce code duplication and make maintenance easier. Input validation and error handling are considered to prevent unexpected application failures. The code follows common programming conventions and maintains a consistent structure throughout the project. Overall, the source code is understandable and organized, although additional documentation and refactoring can further improve its quality.

b) Identify strengths and areas for improvement.

Strengths:

- Code is logically organized.
- Meaningful names are used for variables and functions.
- Functions improve modularity and reusability.
- Code formatting is consistent.

- Basic error handling and validation are implemented.
- Git helps maintain different versions of the source code.

Areas for Improvement:

- Add more comments for complex logic.
- Improve documentation for individual modules.
- Reduce repeated code wherever possible.
- Add stronger exception handling.
- Follow a consistent coding standard throughout the project.
- Add automated testing for important functions.
- Improve code optimization where performance can be increased.

4. Version Control Evaluation

a) Analyze the Git repository by reviewing commit history, branching strategy, commit messages, and repository maintenance practices.

The Git repository was evaluated by examining the commit history, branches, commit messages, and general maintenance practices. Git provides a complete history of changes made to the project. Each major modification can be tracked through individual commits. Meaningful commit messages such as Initial project setup, Added user module, and Updated validation help identify the purpose of changes. A suitable branching strategy can be used to keep the main branch stable while new features are developed in separate branches. After testing, completed changes can be merged into the main branch. The .gitignore file can be used to prevent unnecessary files such as temporary files, build files, and environment-specific files from entering the repository. Regular commits and proper branch management improve project traceability and repository maintenance.

b) Explain how version control supports collaborative software development.

Version control allows multiple developers to work on the same project without losing previous changes. Developers can create separate branches for different features or fixes and merge the completed work into the main project. Git maintains a history of changes, making it possible to identify when and why a modification was made. If an incorrect change is introduced, an earlier version can be restored. Merge and conflict-resolution features help combine contributions from multiple developers. Git also improves accountability because each commit records the contributor and the changes made. Therefore, version control improves collaboration, change management, reliability, and overall software development efficiency.

5. Code Testing and Validation

a) Demonstrate that the application functions correctly through appropriate testing.

The application is tested to verify whether each major functionality produces the expected result. Functional testing is performed by providing valid and invalid inputs to different modules. Unit-level testing can be used to verify individual functions, while integration testing verifies communication between different modules. System testing checks the complete application workflow. Input validation is tested using incorrect, empty, and boundary values. The observed output is compared with the expected output to determine whether the application works correctly. Any errors identified during testing are corrected and the application is tested again. This process helps ensure that the final implementation satisfies the project requirements.

b) Present test cases, execution results, and screenshots as evidence.

Test Case	Input/Condition	Expected Result	Actual Result	Status
TC01	Valid user input	Correct output generated	Correct output generated	Pass
TC02	Invalid input	Error message displayed	Error message displayed	Pass
TC03	Empty input	Input validation message	Validation message displayed	Pass
TC04	Valid data submission	Data processed successfully	Data processed successfully	Pass
TC05	Incorrect data	System rejects invalid data	Invalid data rejected	Pass

Evidence to Include:

- Screenshot of application execution.
- Screenshot showing successful output.
- Screenshot showing input validation/error handling.
- Screenshot of test execution results.
- Screenshot of Git repository and commit history.

6. Repository Documentation

a) Evaluate the completeness of the project documentation, including the README, installation instructions, usage guide, and dependencies.

The repository documentation provides essential information required to understand and execute the project. The README file should contain the project title, problem statement,

objectives, features, technologies used, installation procedure, usage instructions, and testing information. Installation instructions should clearly explain the software and tools required to run the application. The dependency file should list the required libraries and packages. A usage guide should explain how users can start and operate the application. Documentation is important because it enables new developers to understand the project without depending on the original developer. Proper documentation also improves project maintainability and makes deployment easier.

b) Suggest improvements to enhance usability and maintainability.

The documentation can be improved by providing a detailed project overview and clear installation steps. Screenshots can be added to explain important application features. The README should include the required software versions and dependencies. Common errors and their solutions can also be documented. API or module documentation can be included if the project contains multiple services or modules. Instructions for running tests should be provided so that developers can validate the application easily. A contribution guide can also be added to explain Git branching and commit practices. These improvements make the repository more user-friendly and easier to maintain.

7. Code Review Findings and Recommendations

a) Summarize the overall code review findings.

The overall code review shows that the project has a structured implementation with clearly defined functionality. The repository organization makes the source code and supporting files easy to locate. The source code is reasonably readable and modular, with functions and modules used to separate different operations. Git provides effective version control and maintains the history of project changes. Testing confirms that the major application functions work according to the expected requirements. The documentation provides the basic information needed to understand and execute the project. However, improvements can be made in automated testing, detailed documentation, error handling, code optimization, and repository maintenance. Overall, the project follows good software development practices and provides a suitable foundation for further enhancement.

b) Recommend improvements related to code quality, repository management, documentation, testing, and future enhancements.

Recommended Improvements:

- **Code Quality:** Follow consistent coding standards and improve modularity.
- **Code Documentation:** Add comments and documentation for complex functions.
- **Repository Management:** Use meaningful commit messages and feature-based branches.
- **Documentation:** Improve README content with installation and usage instructions.

- Testing: Increase test coverage and introduce automated testing.
- Error Handling: Add comprehensive exception handling and input validation.
- Security: Protect sensitive information and avoid storing credentials in the repository.
- Performance: Optimize frequently executed operations and reduce unnecessary processing.
- Deployment: Use Docker or similar containerization methods for consistent deployment.
- Future Enhancements: Add new features based on user requirements and continuously improve the system.

Expected Deliverables

- Code Review Report (10–15 pages)
- Git repository link
- Screenshots of repository structure and commit history
- Code review observations with examples
- Test execution screenshots
- Recommendations for improvement

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Problem Analysis & Repository Organization(15%)	Clearly explains the problem, solution, and repository structure with logical organization and justification.	Good explanation with minor omissions.	Basic explanation and repository organization.	Incomplete or poorly organized repository.
Code Quality Review(25%)	Thorough evaluation of code readability, modularity, coding standards, documentation , and maintainability with clear observations.	Good code review with minor gaps.	Basic review with limited analysis.	Superficial or inaccurate code review.
Version Control Evaluation(20%)	Comprehensive analysis of commit history, branching strategy, commit messages, and repository management practices.	Good evaluation with adequate explanation.	Basic evaluation with limited evidence.	Poor or incomplete version control analysis.
Testing & Validation(15%)	Demonstrates comprehensive testing with appropriate	Good testing with minor omissions.	Basic testing with limited evidence.	Inadequate or missing

	test cases, execution results, and supporting evidence.			testing and validation.
Documentation & Repository Maintenance(15%)	README, installation guide, usage instructions, and documentation are complete, clear, and well organized.	Documentation is mostly complete.	Basic documentation with missing details.	Poor or insufficient documentation.
Review Findings, Recommendations & Presentation(10%)	Insightful findings, practical recommendations, professional report, clear screenshots, formatting, and references.	Good recommendations and presentation.	Basic recommendations with acceptable presentation.	Weak conclusions and poor presentation .

Criteria	Mark
System Requirement Analysis (30%)	/5
Selection of Software Architecture (25%)	/5
Architecture Diagram and Component Design (20%)	/5
Component Interaction and Quality Attribute Analysis (15%)	/5
Architectural Justification and Technical Presentation (10%)	/5
TOTAL	/25